LudoGraph 文献综述：面向游戏规则的可验证知识图谱

LudoGraph 研究团队

2026 年 10 月 10 日

1 摘要

LudoGraph 试图回答一个横跨游戏设计理论、神经符号信息抽取、知识图谱工程、图查询编译与形式化验证的问题：**能否把商业游戏策划文档中的规则事实，保守地、可追溯地、可验证地转化为一个可计算的领域知识图谱，并在此基础上对经济死锁、状态机冲突与数值越界给出可回放的反例或带假设的证明？** 本文围绕这一问题，综合既有研究成果，比较相关学术谱系，而非逐篇罗列结论。

综述的主要判断有四点。**第一，组件层面已充分成熟，系统层面的整合仍是空白**。游戏设计理论提供了分层分析工具（MDA、Sicart 机制定义、GOP、GDL/GDL-III、Temporal Frames）；神经符号抽取提供了从约束解码到本体校验的完整技术栈；属性图数据库与 NL2Cypher 已有公开基准与官方工具；形式化验证在 Petri 网、时序逻辑、抽象解释、SMT/CHC 和随机博弈上都有数十年理论积累。但这些组件分别针对短句、固定本体、干净输入或手工形式化模型，**没有一个公开、可复现、专门面向商业游戏策划文档的本体化抽取与验证流水线**——该 "不存在" 的强版本仍须谨慎对待。

第二，"格式正确" 与 "事实正确" 与 "模型正确" 是三个必须被分离的层次，也是当前文献最容易被合并的混淆点。约束解码保证 JSON/语法合法，本体 validator 保证类型与 domain/range 合法，证据绑定保证 claim 可回指原文，形式化验证保证结论对**形式化后的模型**成立。任何一层都不能替代另一层；求解器对错误编码的模型会给出完全正确的错误答案。这是本文对相关研究的横向综合结论，也是该研究立论的核心。

第三，游戏规则文本的特殊性恰恰落在现有基准的盲区上。Text2KGBench 使用短句与固定本体，GraphRAG/LightRAG 的社区摘要会损失限定词，通用三元组抽取会把 "当 X 且非 Y 时每回合一次地 Z" 压扁为永久 `X causes Z`。条件作用域、模态、否定、周期与重置语义、概率与权重、版本覆盖与例外优先级、玩家能动性量词——这些维度在通用 KGC 与通用形式化文献中要么未被评测，要么被假设为已给定。游戏领域因此存在可发表的结构性空白，其形态是 "规则语义的可验证表达"，而非 "再做一个通用 OpenIE prompt"。

第四，LudoGraph 最有研究价值的切入点不是求解器接入，而是 "带证据与版本的保守编译"。既有研究提出的五个创新候选中，候选一（带证据与不确定性标签的规则保守编译器）、候选三（版本演化的精化保持与反例迁移）、候选二（守恒—可达—策略分层不变量综合）最有可能形成硬贡献；本文认为它们的共同理论内核是**为自然语言到形式模型的翻译定义可证明的 soundness 契约，并让不确定性、版本和冲突成为一等语义而非事后修补**。

本综述同时明确列出未知与风险：自然语言编译正确性是最高风险环节；"经济可持续""可玩性" 缺乏统一定义；公开游戏形式化案例不足以证明大型商业系统的可扩展性；国内相关工作覆盖不充分（`[unverified]`）；所有 "毫秒级" 性能表述都只是待测工程目标而非已证实能力。综述最后给出研究合法性论证、具体切入点、可证伪的实验设计与评测协议建议，以及一份保留可核验链接

的参考文献表。

关键词：游戏规则本体；神经符号抽取；证据绑定；属性图；NL2Cypher；形式化验证；可判定性；保守编译；版本演化；可玩性代理指标

2 背景与范围

2.1 问题从何而来

游戏工业的规则系统具有一种结构性的工程困境。一方面，现代服务型游戏的规则集合（经济系统、掉落表、任务链、状态机、技能冷却、数值成长曲线、限时活动、交叉依赖）在复杂度上已经不亚于一个中型软件系统；另一方面，这些规则的**权威来源**往往不是可执行代码，而是自然语言策划文档、Excel 配置表、流程图、伪代码和口头约定。相关研究 引用游戏需求工程研究指出，游戏开发具有快速迭代、跨利益相关者和 gameplay requirements 难以显式表达的特点（A Holistic Look at Requirements Engineering in the Gaming Industry；How practitioners approach gameplay requirements?）。这意味着 “规格” 与 “实现” 之间的语义鸿沟在游戏领域比在传统软件中更宽，而不是更窄。

传统 QA（人工测试、自动化脚本、灰度发布、线上监控）在 “发现已知问题” 上有效，但在 “证明未知规则组合不违反约束” 上存在根本局限。形式化验证在原则上可以填补这一缺口，但它要求一个可计算、无歧义、结构化的形式模型；而策划文档恰恰缺乏这种模型。于是问题被拆成三个必须串联的子问题：

1. **抽取问题：**如何从非结构化/半结构化文本中抽出**原文明确说出**的规则事实，而不是让模型补写一个 “合理的世界”？
2. **表示问题：**用什么样的本体、存储模型与查询接口承载这些事实，使其既能被审计、又能被机器查询？
3. **验证问题：**如何把这些事实编译成可判定、规模可控的约束问题，并正确解释 `sat` / `unsat` / `unknown`？

既有专题调研 分别覆盖了这三个子问题的理论基础、技术选型与形式化方法。本综述的任务是把它们组织成一张可批判、可比较的学术地图。

2.2 范围界定

在范围内：游戏设计理论与游戏本体；本体受限的神经符号抽取与证据绑定；知识图谱存储与查询路由；形式化验证方法论与可判定性；上述方向之间的接口、冲突与研究空白。

不在范围内：LudoGraph 的具体技术选型决策；代码实现；对既有设计文档的修改建议；对商业产品的性能承诺。本文的定位是研究地图与合法性论证，不是工程方案书。

一个必须预先声明的边界：本文所综述的“形式化验证”指**验证规则/数值系统本身**，而非验证某个 AI/代理的行为。相关研究 明确区分了两者：前者量化所有合法玩家行动、所有随机结果和版本路径，检查规则是否存在死锁、不可达目标、资源枯竭或无限刷取；后者固定一个策略并检查其是否遵守约束或能否获胜。这一区分至关重要，因为二者的量词方向（`forall` 玩家行动 vs `exists` 玩家策略）必须在中间表示（IR）中显式保存，且“证明一个 AI 不作弊”不能推出“规则对所有玩家都有可行动路径”。

2.3 检索方法与证据分级

本综述的输入是七份已完成的范围性调研（既有专题调研），它们各自声明了检索日期、检索范围与关键词。我们需要在此**批判性地审视这一检索基础本身的局限**，因为综述的可靠性不能超过其输入：

- **语言偏置：**上游检索以英文公开资料为主。相关研究 与 相关研究 都明确指出未对中国知网、万方等中文数据库做完备系统综述，因此“国内无相关工作”的结论被显式标为 `[unverified]`。
- **时效偏置：**相关研究 指出 2026 年的新论文、引用量与仓库活跃度尚未形成稳定索引，相关具体结论标为 `[unverified]`。这意味着 相关研究 中 2025 年之后的论断具有天然的时间脆弱性。
- **可复现性偏置：**多份文档记录了开源仓库的 stars/issues/许可证状态，但 相关研究 明确指出 stars 是仓库指标而非论文质量结论，且许可证与维护状态随版本变化。
- **“不存在”命题的不可证伪性：**多份文档都出现“未发现某个 benchmark/系统”的表述。严格说，检索的否证永远是不完全的；因此这些命题的强版本一律标 `[unverified]`，只有“在本次检索范围内未发现”这一弱版本是事实。

基于上述局限，本文对全部论断采用三**级证据分级**：

级别	含义	本文的处理方式
事实（F）	可由上游引用的论文、官方文档或仓库直接核验的陈述	保留可核验链接，陈述其具体内容

级别	含义	本文的处理方式
学界共识（C）	多份独立来源一致支持、且上游明确标注为共识的判断	说明共识的依据来源，不夸大为定理
本次推断（I）	上游基于事实对 LudoGraph 的研究判断，或本文的综合判断	显式标注推断性质，给出依据，不表述为已证实结论
未核实（U）	上游标注 `[unverified]` 或无法由公开资料直接确认	原样保留标记并说明不确定性来源

本综述的五条写作原则（对应 5C）：全面性（Comprehensive，覆盖 既有专题调研 全部实质结论）；简洁性（Concise，以论题组织而非以文档组织）；连贯性（Coherent，各章共享同一分析框架）；批判性（Critical，显式比较方法边界与相互矛盾）；一致性（Consistent，事实/共识/推断分级贯穿全文）。

2.4 六步方法论

本文按以下六个步骤组织，这也是后续章节的顺序：**（1）梳理学术谱系与主题分类**（第 2 章）；**（2）逐主题深读并提取可比较的维度**（第 3–6 章）；**（3）横向比较与争议识别**（第 7 章）；**（4）定位研究空白**（第 8 章）；**（5）论证 LudoGraph 的研究合法性与切入点**（第 9 章）；**（6）风险、未知与可证伪的验证计划**（第 10 章）。最后给出结论（第 11 章）与参考文献（第 12 章）。

3 学术谱系与主题分类

3.1 一条主线，四个谱系

LudoGraph 的完整论证链是 “文本 → 事实 → 图谱 → 查询/编译 → 证明/反例”。沿这条链可以识别出**四个相对独立的学术谱系**，它们各自有独立的成熟度曲线、评价标准和内在争论。把它们混为一谈是文献综述中最常见的失败模式：例如把 “LLM 能生成合法 JSON” 当作 “抽取可靠”，或把 “Z3 能求解约束” 当作 “规则可被验证”。

谱系	核心问题	代表来源（本综述输入）	成熟度
A. 游戏理论与本体论	游戏系统由哪些不可再分的对象、动作、状态、规则构成？	MDA、Sicart、GOP、GDL/GDL-III、Temporal Frames、agency-centered ontology	理论成熟，但无统一的“可抽取本体”标准
B. 神经符号抽取与知识图谱构建	如何从文本保守地抽取受本体约束的事实并绑定证据？	Text2KGBench、EDC、ChatIE、KARMA、Wikontic、约束解码、CoT、FEVER/ERASER	组件成熟，领域化与证据保真仍开放
C. 图存储、查询与路由	用什么模型承载事实，如何把自然语言问题编译为可执行查询？	Neo4j/Kùzu/FalkorDB/AGE、Text2Cypher 基准、llama.cpp 量化基准	工程成熟，跨方言与延迟指标不透明
D. 形式化验证与可判定性	如何把规则编译为约束问题并正确解释结果？	Petri 网、模型检查、ATL、PRISM-games、抽象解释、CEGAR、CHC、SMT-LIB	理论成熟，游戏语义编译开放

一个跨谱系的观察（I）：四个谱系在 2023–2026 年出现了一个共同的转向：**不再追求单一端到端模型，而是走向“生成候选 → 检索/本体对齐 → 验证/融合 → 图检索或求解”的分层组合**。相关研究 依据 EDC、KARMA、Wikontic、SymAgent、KG-Prover 的共同结构得出这一判断，并强调它不是单篇论文的口号。这一转向对 LudoGraph 具有直接意义：项目的架构（非对称神经-符号流水线）与当前研究前沿的方向一致，而不是逆流。

3.2 谱系 A：从 MDA 到可计算规则语言

游戏设计理论谱系内部本身存在一个从“分析框架”到“可执行语言”的光谱（详见第 3 章）。关键的分类维度是：**该理论是否提供可判定的状态转移语义？**

- MDA（Hunicke, LeBlanc & Zubek）把游戏拆为 Mechanics / Dynamics / Aesthetics，是一个**分析性**分层，不提供形式语义。
- Sicart 把 game mechanics 定义为“由代理人调用、用于与游戏状态交互的方法”，并区分 mechanics 与 rules，提供了**面向代理人**的精确化。

- GOP（Game Ontology Project）是一个**持续演进的层级概念框架**，自我定位为分析工具而非唯一分类。
- GDL / GDL-III 是**形式语言**，把游戏建模为有限状态图，具有明确的状态转移语义。GDL 原始范围是有限、离散、确定性、完全信息游戏；GDL-III 扩展到不完全信息与认知游戏。

这一光谱的重要性在于：**LudoGraph 的本体设计必须同时服务于“分析”与“编译”两个目的，而这两个目的对精确性的要求不同**。分析导向的本体（GOP）允许模糊与演进；编译导向的本体（GDL）要求无歧义。相关研究 的处置建议（保留 15 个核心标签、收窄 `Effect`、降级 `Condition`、废弃开放性 `Concept`、新增 `Agent/Goal/State/TimeFrame/RandomProcess`）本质上是在向“可编译”方向移动本体，同时又保留分析层的表达能力（`System`、`Level`、`Trait`）。

3.3 谱系 B：抽取路线的三档约束强度

神经符号抽取谱系可按**约束发生的时机与强度**分为三档，这一分类比“pipeline vs 端到端”更有判别力（详见第 4 章）：

1. **提示式软约束：**字段定义与示例放入 prompt，模型自由生成 JSON。实现最简单，但只能形成软约束。
2. **API 级结构化输出/约束解码：**工具调用声明 JSON Schema（OpenAI Structured Outputs 的 `strict: true`），或在 token 生成时用 grammar/CFG/正则限制可行 token（Outlines、Guidance、LM Format Enforcer、llama.cpp grammar）。
3. **本体级与证据级硬约束：**在解码之后由确定性 validator 检查标签白名单、domain/range、连续 quote、限定值 raw/unit、certainty 与冲突规则。

相关研究 的核心判断是：这三档解决的是**不同层次**的问题，不能相互替代；最稳妥的组合不是让模型直接生成图查询，而是“版本化、封闭的中间 JSON 合同 + 确定性 validator”。这一判断得到 相关研究 的独立支持：Text2KGBench、EDC、维护短文本实验和 KG 构建综述都支持“LLM 适合候选发现但不能单独充当事实数据库”。

3.4 谱系 C：存储模型与查询编译的解耦

图存储谱系的分类维度是**数据模型（LPG vs RDF）**与**部署模型（嵌入式 vs 服务端）**；查询路由谱系的分类维度是**端到端生成 vs 意图路由 + 编译**（详见第 5 章）。两章的关键综合点是：**LudoGraph 的 claim 模型（subject、predicate、object、限定值、certainty、证据、来源）是一个与具体数据库无关的抽象合同，存储与查询都应通过适配层实现**。这既是工程选择，也是研究上的可迁移性要求：只有把“抽取合同”与“物理投影”分离，才能让验证层的语义不被某个数据库的特性所污染。

3.5 谱系 D：验证方法的四种量词结构

形式化验证谱系可按**被量化对象的类型**分类（详见第 6 章）：

- **纯结构（无量词）：**图遍历、SCC、可达性、one-hot、类型检查 → 有限图上的线性或多项式算法。
- **全称路径（∀ 路径）：**LTL 安全性/活性、抽象解释的过近似 → “所有路径都不越界”。
- **存在路径（∃ 路径）：**有界模型检查的反例、Petri 网可达性 → “存在一条可执行路径导致违规”。
- **交替量词（∃ 策略 ∀ 环境）：**ATL/ATL*、随机博弈 → “存在玩家策略使目标无论环境如何都成立”。

这一分类是理解 相关研究 与 相关研究 关系的关键。相关研究 主要停留在前两类（图算法 + 有界 SMT），相关研究 把视野扩展到全部四类并给出了分层方法。本文认为，这一扩展不是 “更全面”，而是**必要的精确化**：因为游戏规则中的 “可持续”“可玩”“不会卡死” 在数学上分属不同的量词结构，把它们统一压成 “是否存在死锁” 会丢失语义。

4 主题一：游戏理论与本体

4.1 四套理论各自回答不同问题

相关研究 的核心发现是：**游戏设计理论提供的是不同观察层，而不是一套节点清单**。这一发现值得展开为方法论原则。

MDA 的价值在于说明 “规则/动作如何导致状态变化，再影响体验”。但 MDA 的 Aesthetics 是玩家与系统交互时产生的期望情感反应（Hunicke, LeBlanc & Zubek, MDA），**不是一条可从策划文本直接抽取的玩法事实边**。相关研究 据此建议：不应把 Aesthetics 当作 LudoGraph 的普通节点标签；否则会混淆设计意图与系统事实。这一判断直接呼应 相关研究 的核心警告——求解器只对形式化模型的可靠性负责，若把设计意图（“玩家应当有选择”）与运行时事实（“系统允许这些动作”）混为一类，后续验证的语义基础就是错的。

Sicart 将 game mechanics 定义为 “由代理人调用、用于与游戏状态交互的方法”，并明确区分 mechanics 与 rules（Defining Game Mechanics）。这为 `Action`、`Rule`、`Condition`、状态变化之间的分工提供了比 “所有机制都叫 System” 更精确的依据。特别是，Sicart 明确允许代理人是人类或计算机，这支持将 “谁执行”（Agent）与 “能执行什么”（Action）分离。

GOP 自我定位为 “从多个具体游戏分析中抽象出的、持续增长的层级概念框架，而不是唯一正确的分类”；其顶层包括 Interface、Rules、Entity Manipulation、Goals（GOP 官网）。GOP 的价值在于提醒 `Goal`、玩家行动、规则、空间与交互是独立的分析维度，但它**不要求也不保证**逐字复

制其层级。

GDL / GDL-III 与前三者的性质不同：它是一个形式语言。官方说明明确列出 `role`、`base`、`action`、`init`、`control`、`legal`、`goal`、`terminal` 等语义，并用规则描述动作对下一状态的影响（GDL Overview、GDL Specification）。GDL 原始范围是有限、离散、确定性、完全信息游戏；GDL-III 扩展到不完全信息和认知/知识游戏（IJCAI: GDL-III）。

4.2 批判性比较：为什么不能只选一套理论

一个自然的问题是：LudoGraph 能否直接采用 GDL 作为本体？ **答案是否定的，而且理由具有普遍的方法论意义**。

1. **GDL 是语言而非抽取本体**。GDL 描述的是**已经形式化的游戏**；它不解决 “从自然语言策划文档中识别什么是规则” 这一抽取问题。它规定了 `role`、`legal`、`goal`，但没有规定中文策划文本中的 “限时活动期间每日首充双倍” 应映射到哪些 GDL 谓词。
2. **GDL 的确定性假设与商业游戏不符**。原始 GDL 假设确定性；商业游戏普遍含概率掉落、暴击、匹配。GDL-III 扩展到不完全信息，但其认知语义仍不等同于 “策划文档中的概率与权重表”。
3. **GDL 缺少版本与例外语义**。相关研究 指出，大型游戏规则系统包含 “版本覆盖和异常例外”，而这是形式化游戏语言传统中较少被作为一等语义处理的部分。
4. **GOP 缺少可判定语义**。反向地，GOP 的分析框架无法直接编译为约束。

因此（I）：LudoGraph 的本体设计需要一种 “双层结构”——上层采用分析导向的标签（服务于召回与人类可读性），下层为每个 claim 提供可编译的受控限定字段（服务于验证）。相关研究 建议的最小 claim 字段 `agent`、`goal`、`temporal_frame`、`probability`、`outcome_set`、`condition` 正是这一双层结构的具体形式。这不是从任何单一理论推出的结论，而是本文对 相关研究 建议的理论重读。

4.3 时间、能动性与经济作为独立维度

相关研究 识别出三个不能由普通实体标签替代的维度，这是该章最具比较价值的部分：

时间。Zagal 与 Mateas 的 Temporal Frames 将游戏时间视为事件之间的关系，区分 real-world time、gameworld time、fictive time 和 coordination time；回合、轮次、tick、延迟行动和同时行动都属于协调时间语义（A Unifying Framework for the Analysis of Game Temporality）。这直接支持 相关研究 的警告："每回合一次""服务器 tick""现实时间冷却""赛季结束重置" 不可在 IR 中都压成一个整数计数器。**本文强调：**这一条在文献综述中的分量被低估了——它意味着时间语义的丢失不是 "精度损失"，而是**语义类别错误**，会导致验证层对完全不同的规则给出相同的（错误）结论。

能动性。2023 年的 agency-centered ontology 工作明确以 mechanics、玩家能动性及两者关系为建模目标（Mitchell et al., Towards an Agency-centered Ontology of Game Mechanics）。这支持把玩家、AI、系统、队伍角色和它们的目标/意图作为 agent 维度，而不是把 "谁做了动作" 隐含在 `HAS_ACTION` 中。**本文补充（I）：**能动性维度在验证层的对应物正是 相关研究 的 "量词方向" 问题——谁的选择被全称量化，谁的存在性被断言。本体上的 `Agent` 缺失会直接导致验证上的量词错误。

经济。经济设计实践通常以 sources/faucets 和 sinks/drains 追踪资源产生与消耗（GameDeveloper, A Practical Guide Into Creating a F2P Game Economy）。这与 `PRODUCES`、`CONSUMES`、`COSTS`、`RESTORES` 的方向相符，但 相关研究 明确指出经济验证还需要周期、流量、所有者、汇率、库存和随机掉落。**本文认为（I）：**这解释了为什么 相关研究 把 "存在一个资源流" 与 "存在合法操作序列" 区分为两种不同性质——前者只需要净产出的 LP 可行性，后者需要库存非负、前置条件与可执行顺序，而后者才是经济死锁的真正对象。

4.4 本体的一个直接可复核问题

相关研究 报告了一个具体的、可复核的计数不一致：设计文档标题与工单称有 **25 个关系类型**，但文档表格逐项数得出 **24 个唯一关系类型**。文档明确说明这是 "本地设计文档的可复核计数，不是理论判断"，并建议在进入实现前解决。**本文将其记录为一个事实（F）**，因为它可以独立复核，且它揭示了一个方法论要点：**本体文档的计数与抽取白名单必须来自单一权威来源**，否则 schema 版本与抽取白名单会出现不一致。这不是游戏理论问题，而是知识图谱工程的治理问题，但它在文献综述中具有代表性——它示范了 "看起来像理论问题的缺口，实际上是规格同步问题"。

4.5 本章小结

游戏理论谱系为 LudoGraph 提供的是**分析语汇与正确性标准**，而非现成本体。最重要的三条认识是：分层是必要的（MDA/Sicart/GOP/GDL 都以不同方式支持规则、动作、状态、目标、体验的分层）；可判定语义是编译的必要条件（只有 GDL 类语言提供）；时间、能动性、经济、信息状态是需要独立建模的维度。本章的主要推断是：LudoGraph 需要 “分析层 + 可编译限定层” 的双层本体，而 相关研究 建议的最小 claim 字段是这一结构的雏形。

5 主题二：神经符号抽取与证据绑定

5.1 三个层次的分离与它们经常被混淆的方式

相关研究 开篇提出的四层约束（结构、本体、证据、语义/业务审核）与 相关研究 提出的三层（格式、结构/本体、事实/证据）在实质上是同一分类。本文将其统一为三**个必须分离的层次**，并指出每一层被误当作另一层的典型失败模式：

层次	保证什么	不保证什么	典型误用
格式层	JSON/Schema/语 法 合法	字段语义	“structured output 成功 = 抽取成功”
本体层	标 签/谓 词/domain-range 合法	事实是否存在于文本	“domain-range 合法 = 关系为真”
事实层	断言被原文 span 支持	推断是否正确、模型是否正确编码	“quote 相关 = quote 蕴含 claim”

这一表格是本综述对神经符号抽取文献的核心综合（I）。它解释了为什么 “约束解码 + 本体校验” 的堆叠不能自动给出可信 KG：每一层只能排除自己那一类错误。相关研究 明确指出，即使 quote 连续且相关，也不代表它支持 claim 的全部方向、条件和数量；需要规则 + NLI/人工复核，而 NLI 本身可能幻觉。

5.2 约束解码：成熟度最高，也最容易被误读

当前实践的三档（提示式 JSON、API 级结构化输出、解码时硬约束）中，第三档的技术成熟度最高。代表性开源项目包括 Outlines、Guidance、LM Format Enforcer 和 llama.cpp grammar。相关研究 的判断是：这类方法 “通常能显著降低格式错误，但不自动保证事实与证据正确；复杂 schema 还会带来编译、吞吐和模型兼容性成本”。

本文的批判性补充（I）：文献中存在一种隐含的 “约束越强越可靠” 的直觉，但 相关研究 与 相关研究 都指出这是错的——过强的约束会**降低召回**，并把 “本体未覆盖” 误判为 “文本无事实”。这是一个精度—召回—覆盖三者的权衡，而不是单调改进。相关研究 用一句精辟的表述概括了这一风险：“否则 ontology 约束可能把模型从’ 乱抽’ 变成’ 结构正确地乱抽’。”

5.3 CoT 在抽取中的证据与限制

相关研究 对 CoT 的处置具有示范性的批判价值。一方面，Chain-of-Thought Prompting 和 Self-Consistency 表明分步推理/多路径采样可改善一部分组合推理任务。另一方面，这些结果主要来自数学、常识和推理基准，**不能直接推出 “CoT 一定提升信息抽取”**。抽取还要求忠实于输入，而长篇自由推理会增加模型把常识当作文本事实的机会（Turpin et al., 2023；CoT 综述）。

相关研究 的结论是：默认不把内部 CoT 作为结果字段或证据；仅在需要跨句归因、条件范围或复杂数值解析时启用 “短步骤、隐藏思考、结构化中间字段”；并以消融结果决定是否启用——若 CoT 只提高召回却提高 unsupported-claim rate，应弃用。

本文认为（I）这一处置是整份上游材料中最 “科学” 的设计决策之一，因为它把一个流行技术降级为**可测量的局部优化**，并给出了明确的弃用条件。这与本文第 10 章的风险分析一致：可信性不能来自 “模型更聪明”，只能来自可验证的 span、封闭本体、确定性拒绝和持续回归评测。

5.4 证据绑定：三种方法与一个不可替代的选择

相关研究 比较了三种证据绑定方法：字符/词 token span（输出 `start`、`end`、`quote`，服务端检查 `quote == chunk[start:end]`）、句子/段落引用、检索证据 + 生成（RAG）。其判断是：**span 方法最适合 LudoGraph，因为连续性和可复核性明确**；句子/段落引用适合长文档但不够精细，只能作为定位信息；RAG 降低上下文长度却不能证明每个字段都被证据支持。

可复用的评测资源包括 FEVER、ERASER benchmark、Evidence Inference 和 DocRED supporting facts。但 相关研究 明确指出这些数据集的任务定义不同（FEVER 侧重声明是否被证据支持，ERASER 同时评估 rationale，DocRED 侧重文档级关系），**它们可借鉴指标与人工标注流程，但不等价于游戏策划规则**。

相关研究 推荐的证据合同是一个不可变输入片段的引用：

```
{"quote":"每次使用消耗 2 枚冰晶","start":123,"end":133}
```

validator 应检查：边界合法；quote 是 chunk 的连续子串；subject、predicate、object、quantity、condition **各自**至少有支持片段；不能引用 chunk 外上下文。若只有 “看起来相关” 的证据而没有明确蕴含，应转 `uncovered` 或降低 certainty，而不是创建 confirmed claim。

本文的批判（I）：这一设计的理论意义大于其工程意义。它实际上把 “证据” 从一种**模型输出**重

新定义为一种**可由代码验证的输入引用**。这消除了模型自证的可能，代价是把 “蕴含判断”（quote 是否支持 claim）留给了人/NLI。文献中很少有工作明确承认这一残余风险并把它列为一等未知项；相关研究 把它列为 “证据蕴含难判定”，这是诚实的。

5.5 保守性、拒绝与身份合并：三个被主流基准忽视的维度

相关研究 与 相关研究 一致指出三个维度在通用 KGC 文献中覆盖不足：

1. **保守拒绝（`uncovered`）**。相关研究 建议用 `correctly_rejected / (correctly_rejected + false_positive)` 和 `uncovered recall` 度量，并明确 “不能只追求 recall”。相关研究 建议把 “保守拒绝能力” 列为缺失的评测维度，包括 unknown/uncovered 的 precision、recall 与拒绝错误关系的比例。
2. **实体对齐隔离**。相关研究 建议局部 ID 只在 chunk 内有效，跨 chunk 合并需要独立的 entity-link 流程、候选证据和人工审核，**不能按同名自动合并**。相关研究 独立指出过度 canonicalization（聚类/别名合并）可能把同名角色、不同版本实体和相似规则错误合并。
3. **冲突并存**。相关研究 建议不同来源的矛盾 claim 并存，交给后续审核，不用模型 “综合” 为一个新事实。

本文的综合判断（I）：这三点在文献中不是独立的技术点，而是同一个设计哲学的三个面向——**把不确定性外化为显式的、可审计的状态，而不是在模型内部隐式消化**。这与 相关研究 的 “多结果协议”（`proved` / `counterexample` / `bounded-no-counterexample` / `unknown` / `not-modeled`）在结构上同构。本文因此认为，LudoGraph 的抽取层与验证层共享同一个认识论立场，这是项目内部一致性的来源，也是一个可以写成方法论贡献的东西。

5.6 评测现状：共识与空白

相关研究 第 8 章系统梳理了已有评测（Text2KGBench、MINE-1/KGGen、TACRED/DocRED/SciERC、GraphRAG/HippoRAG 类 QA 基准、KGQA/逻辑查询）并识别出缺失维度。其中最具批判价值的判断是：

- **学界共识（C）：**单一 triple F1 不够，需同时测事实、结构、幻觉和下游任务（依据 Text2KGBench 的七类指标设计及 KG 构建综述）。
- **本次推断（I）：**需要新增证据连续性、条件/模态/否定/时序、数值保真、版本与冲突、本体演化、保守拒绝、图级性质、solver 可验证性、运行成本与稳定性等维度。

一个有代表性的实例：相关研究 指出 KGGen/MINE-1 试图测量 “图从文本保留了多少信息”，而不只看固定 gold triples。这是一个方法论进步，因为传统 exact-match 对开放 schema 不友好。**但本文批判（I）：**MINE-1 的评估主要依赖 LLM-as-a-judge，而 相关研究 自己指出这 “可能偏好流畅

而非忠实输出”，需要人工盲评和 judge 互换/校准。同样的问题出现在 KARMA 报告 “LLM 验证正确率 83.1%” 上。这说明**用 LLM 评估 LLM 在方法上是一个未闭合的循环**，任何以它为唯一证据的结论都应降级。

5.7 本章小结

神经符号抽取谱系的核心张力是**约束强度与召回之间的权衡**，而这一张力无法通过 “更强模型” 消解，只能通过把不同层次的责任分配给不同组件来管理。相关研究 的方案（LLM 提议、schema/grammar 限形、本体 validator 限义、连续证据验真、失败进入 `uncovered`、人工/规则系统裁决）是这一分配的具体形式。本文的主要补充是：这一方案的价值不在于它的组件（每个组件都已有成熟实现），而在于它**明确承认了每个组件无法覆盖的残余风险**。

6 主题三：图谱存储与查询路由

6.1 存储选型：一个被 “基准证据” 约束的工程判断

相关研究 对存储方案的比较有一个值得称道的认识论特征：**它明确拒绝给出未经同一工作负载验证的性能排名**。文档明确指出，推荐 Neo4j Community 作为第一阶段服务端 PoC 的理由是 “需求贴合度及生态成熟度的工程判断，不是经同一工作负载基准证明的性能排名”。这是一个重要的方法论立场，因为图数据库领域的公开对比往往混杂不同版本、配置、硬件和查询分布。

各方比较要点（F）：

- **Neo4j**：原生 LPG、Cypher，关系可携带属性；Community 为 GPLv3，Enterprise 商业许可（Neo4j 开源/许可说明）。
- **Kùzu**：嵌入式 LPG/openCypher 子集，原项目 MIT，但 GitHub 项目已标注 archived；相关研究 明确指出 “原 Kùzu 不宜作为新项目长期依赖”。
- **LadybugDB**：Kùzu 社区 fork（仓库、文档），承接嵌入式方向，但新项目维护风险需观察。
- **Memgraph**：Community 采用 BSL 1.1（非 OSI 开源许可）（仓库及许可说明）。
- **FalkorDB**：兼容 Redis 协议的服务端图数据库（官方文档）。
- **Apache AGE**：PostgreSQL 扩展，Apache 2.0（官网）。
- **Oxigraph / GraphDB**：RDF 三元组/四元组与 SPARQL，而非 LPG/Cypher（Oxigraph 仓库）。

一个跨文档的重要事实（F）：相关研究（路由层）的调研前提包括 “本项目既希望支持 Neo4j，也保留 Kùzu 的存储选项”，而 相关研究 发现 Kùzu 已归档。这构成一个**上游前提失效**的实例：路由层的方言兼容性分析仍具有一般方法论价值，但它针对 Kùzu 的结论需要按 LadybugDB 或其他

候选重新核验。本文将其记录为文档间的时序不一致（I），而非任一文档的错误。

结论翻转条件（I，相关研究 给出）：（1）已有稳定 PostgreSQL 运维且 AGE PoC 满足 SLO → 优先 AGE；（2）确定为单机嵌入式分析 → LadybugDB；（3）需要 RDF/OWL 推理 → 转向 RDF store；（4）Neo4j 许可/TCO 不满足且 FalkorDB/Memgraph 在真实负载 PoC 中胜出 → 重选。这一“预先声明翻转条件”的写法本身值得在其他选型章节推广。

6.2 Claim 存储模型：关系 vs 独立 Assertion 节点

相关研究 的比较表是全文最具工程价值的部分之一。核心取舍是：

维度	Claim 直接成为关系	独立 `Assertion` 节点
读取常见三元组	一跳遍历，路径查询简洁	需经 Assertion 连接，多一跳
证据/来源/限定值	放关系属性；数量/list 能力依产品而异	属性放 Assertion，便于生命周期、审计与多证据集合
重复/冲突	可有多条平行关系，但需应用层幂等键	天然一断言一节点，不同来源可独立共存
修订/废弃	原地更新覆盖历史，生命周期审计脆弱	新建修订版 + `supersedes`，保留可审计历史
聚合/图遍历	适合“当前有效事实”的高频遍历	多一次展开成本，但溯源/冲突/时间切片直接

相关研究 的建议是**把 `Assertion` 作为规范存储**，直接边投影仅作为热点查询的物化优化，且投影不能取代规范断言历史。**本文认为（I）这一取舍的深层理由不是性能，而是语义：**独立 Assertion 节点使“一条断言”成为一等实体，从而让 `status`（`active/deprecated/rejected`）与 `certainty`（`explicit/possible/planned/negated`）在结构上可分离。相关研究 明确警告“两者语义不可混为一谈”，这与 相关研究 的四态 certainty 设计一致。**如果 claim 只能存为关系属性，则“来源撤回”“证据修订”“审计追踪”都会退化为覆盖写，直接违反 相关研究 的“冲突不覆盖”原则。**

6.3 稳定 ID：一个被低估的设计点

相关研究 建议持久 `assertion_id` 由导入端确定性生成：对规范化 tuple `(game_id, system_id, source_id, chunk_id,` 局部 `claim id` 或 `source span, revision)` 做 SHA-256。文档特别警告：**“不要仅哈希三元组，否则不同来源的冲突会被错误合并。”**

本文强调（I）：这是一个把抽取层原则（冲突并存、来源独立）**编码进主键**的设计。它与 相关研究 的“不同来源的说法独立建 Assertion”是同一原则在两个层次上的实现。文献综述中这类“跨

层一致性” 往往被忽视，但它是系统可靠性的实际来源。

6.4 性能证据：诚实的空白

相关研究 在这一点的处理堪称范本。它明确写道：

> 公开基准不构成跨产品可比的写入峰值或毫秒 SLA。……该结果不能推出 LudoGraph 写入吞吐，且仓库原 Kùzu 已归档。

文档给出的 “可验证的预期” 被明确定义为**估算，不是事实/保证**：单机 SSD、常驻服务、索引已热、图数据可容纳于内存的中小型项目，带 namespace/predicate 过滤的索引查 Assertion，再展开 subject/object 的简单查询有机会达到数毫秒量级；多跳遍历、冷缓存、宽范围聚合、写入锁竞争、同步持久化可能显著超过毫秒。文档最后说明：“不能在缺少数据规模与硬件时诚实承诺高并发写入数值。”

本文认为（I）这段表述应被视为整个项目的 “证据纪律” 范例。它区分了 “理论预期” 与 “实测结论”，并给出了可复现 PoC 的具体设计（预计实体/断言数量与 10 倍增长数据、批量导入、并发追加、按游戏/系统/predicate 查询、冲突审计、1–3 跳图查询；报告 p50/p95/p99、吞吐、CPU/RAM、索引/存储体积、尾延迟）。

6.5 查询路由：公开基准给出的硬数字

相关研究 提供了本项目中少见的、可核验的量化证据。论文 A Benchmark Dataset towards Text2Cypher Task 报告了跨 11 个图数据集的 27,529 个自然语言/Cypher 对, 训练/验证/测试划分为 18,470/2,000/7,059, 测试域与训练域分离。零样本总执行准确率（原论文表 3）：

模型	参数规模	零样本执行准确率
Granite-3.3-8B-Instruct	8B	26.21%
Llama-3.1-8B-Instruct	8B	30.99%
Phi-4	约 14B	62.01%
Llama-3.3-70B-Instruct	70B	64.69%
Llama-4-Maverick	MoE	70.97%
DeepSeek-V3	MoE	76.75%
Qwen2.5-72B-Instruct	72B	66.32%

这些数字的批判性解读（I）：即使最好的零样本结果（76.75%）也不是可靠的生产级正确率；8B 级零样本仅 26%–31%。论文报告复杂类别与简单类别的差距平均约 37 个百分点，且少样本检

索示例平均只带来约 2.5 个百分点提升，schema 不匹配的示例还可能误导模型。**这意味着 "用一个大模型端到端生成 Cypher" 不是一个可依赖的架构。**

更贴近 SLM 的证据是 Refining Text2Cypher on Small Language Model with RL：以 Qwen2.5-3B-Instruct 为基线，SFT 后使用 GRPO，并增加 key-value 与关系三元组抽取辅助任务；论文报告在**未见 schema** 上达到 85% 执行准确率，相对仅监督微调提高 5.03 个百分点。相关研究 明确警告：**不能直接推断 LudoGraph 的未训练 3B 模型也能达到 85%**。本文完全同意这一保守处理。

6.6 端侧延迟：从 token/s 到端到端的分层记账

相关研究 对 "毫秒级" 的处理同样值得记录。它引用的公开数据是（F）：

- llama.cpp IQ4_NL CPU benchmark：约 1.24B、约 4.5 bpw 量化模型；M2 Ultra 上 Qwen2 3B IQ4_NL 约 1.70 GiB，8 线程生成约 56 token/s，16 线程约 78 token/s（`tg64`）。文档据此计算：仅生成 10–30 个 token 也可能需要约 130–540ms，且尚未计入模型加载、prompt prefill、schema 注入、网络和数据库查询。
- llama.cpp/vLLM 比较：单 RTX 4090、Qwen2.5-3B、单并发、约 2,048 prompt token 和 256 generated token，报告约 90–94ms 端到端运行时间；文档明确警告该测试定义和硬件条件特殊，不能等同于首 token 延迟。

相关研究 由此得出**分层延迟目标**，并明确它们是 "待实测目标，不是当前事实"：高频模板/缓存命中 P50 ≤10ms、P95 ≤30ms；常驻小模型意图路由 P50 ≤50ms、P95 ≤150ms；动态 NL2Cypher P50 ≤300ms、P95 ≤1s。文档给出的关键结论是：

> "毫秒级" 只适用于**缓存、规则、模板和已编译查询**这类快路径；不适用于任意自然语言到 Cypher 的完整冷路径。

本文认为（I）：这一 "总延迟 = 路由 + schema/示例检索 + 查询编译 + 数据库执行 + 结果整理" 的分解，与第 6 章 SMT 性能的分层记账是同一方法。它们共同拒绝了以单一数字（"毫秒级"）替代分层指标的做法，而这正是预调研报告摘要中 "高并发、毫秒级" 表述需要被精确化的地方。

6.7 两步路由 vs 端到端：一个必须标注为推断的选择

相关研究 推荐**意图/槽位路由 → schema-aware 查询编译**的两步架构，并诚实标注这是 "基于风险和可验证性的工程推断，而非已被同一基准严格证明的定理"。本文赞同这一标注，并补充本文自己的理由（I）：公开基准实际测的是**端到端** Text2Cypher 的准确率，因此 "两步更好" 缺少直接的对等实验支持；两步架构的证据来自间接推理（分类器可枚举可测、编译器输入更短、IR 可跨方言复用、可降级到云端）。**这恰好是一个可发表的研究空白：两步与端到端的受控对比实验。**

6.8 本章小结

存储与路由谱系的关键综合是：**它们都应当被设计为“可替换的实现基线”，而不是系统语义的一部分**。相关研究 的“抽取合同与物理存储分离”、相关研究 的“统一 IR → 后端能力探测与方言编译”，在结构上是同一原则。本文的主要批判是：这两份文档的量化证据（Text2Cypher 准确率、llama.cpp token/s）虽然可核验，但都**来自与 LudoGraph 不同的分布**，因此只能支撑“不应承诺什么”，不能支撑“可以达到什么”。

7 主题四：形式化验证与可判定性

7.1 从 相关研究 到 相关研究：一次必要的理论升级

相关研究 与 相关研究 覆盖同一领域，但抽象层级不同。比较二者可以清晰看到一次方法论升级：

维度	相关研究	相关研究
主问题	“把图交给 Z3” 如何工程化	验证对象是 “演化中的规则系统”
方法	图算法 + 区间分析 + 有界 SMT	六层分层（结构/抽象数值/离散动态/约束精化/无界证明/随机博弈）
量词	主要 ∃（反例）与有限 ∀	明确区分 ∀ 路径、∃ 路径、∃ 策略 ∀ 环境
时间/概率	提到但未系统化	时间/回合语义与随机博弈作为独立层
版本	未作为一等语义	候选三：版本演化的精化保持与反例迁移
结果协议	`safe within assumptions` / `counterexample` / `unknown`	六类：`proved` / `counterexample` / `bounded-no-counterexample` / `unknown` / `not-modeled`（及 `spurious-counterexample` 等）

本文认为（I）相关研究 不是 相关研究 的重复，而是把 相关研究 的“三类检查”（经济死锁、状态机冲突、数值极值）重新放置到正确的理论坐标系中。特别是，相关研究 已经指出“最高风险

不是 Z3 API 本身，而是从知识图谱 claim 到形式化语义的正确、保守编译”，相关研究 把这一洞察发展成完整的 “保守编译” 研究纲领（候选一）。

7.2 三类检查的形式化：相关研究 的具体方案

相关研究 对三类检查给出了可操作的编码草案（I，文档明确说明 “不代表已在 LudoGraph schema 上验证”）：

经济死锁。变量：资源 `r ∈ R`、操作 `a ∈ A`、库存 `x[r,t] ∈ Int`；`fire[a,t] ∈ Bool`；配方系数 `consume[a,r]`、`produce[a,r]`。约束包括初始库存、可执行性（`fire[a,t] ⇒ x[r,t] ≥ consume[a,r]`）、资源转移 `x[r,t+1] = x[r,t] − Σ fire·consume + Σ fire·produce`，以及死锁反例编码（对每个 `a`，`¬enabled[a,k]` 且 `¬goal[k]`）。相关研究 特别警告：**“循环可能可持续，死锁取决于初始资源、产消量、操作前置条件和可执行顺序”**，且 SCC 或资源依赖环**不是**死锁证明。文档还警告不要 “将玩家可主动等待误判为死锁”。

状态机冲突。用 one-hot 状态编码，检查 `s[m,q,t] ∧ s[m,q',t]`（`q≠q'`）是否可满足；不可达性通过逐步增加 `k` 检查 `s[m,q*,k]`。文档的关键警告：**固定深度 UNSAT 不可推广到无界**；多状态机同步时 “为共享事件、锁、优先级和更新原子性建立显式联合转移；否则可能制造不真实的冲突或遗漏竞争”。

数值极值。每个数值属性按运行时语义选择 `Int / Real / BitVec(w)`；检查 `v_i(t)<L_i ∨ v_i(t)>U_i`。文档强调：“检查溢出：若程序用 `w` 位整数，则按位向量运算验证结果与期望数学整数之间的差异；不应仅用无界 `Int` 推断机器溢出。” 以及 “’ 爆炸’ 不是自带数学定义的性质”。

本文的综合（I）：这三段编码的共同结构是**先定义性质，再定义模型**。这与 相关研究 的 “先定义性质词典，再定义模型” 完全一致，也解释了为什么 相关研究 反复警告 “不要把有限深度无反例标成全局安全”——因为性质的范围（有界/无界、假设集合）没有先被定义清楚。

7.3 SMT 性能：为什么不能按约束条数承诺延迟

相关研究 与 相关研究 在性能问题上给出了罕见详尽的否定性证据（F）：

- TU Delft 对 20 个实数约束基准的研究中，部分 QF_NRA 实例平均耗时为秒级（MBO 默认策略非超时实例均值约 50.47 秒，另一策略约 12.99 秒），另一些基准表现截然不同（论文）。
- Z3 官方 `Programming Z3` 说明增量查询依靠上下文复用，但增量模式可能改变求解策略；现实案例中，加入 `push` 后同一类实例从 0.03 秒升至 151.32 秒（Programming Z3；Z3 issue #1141；Z3 issue #7441）。

相关研究 据此得出："SMT 的运行时间没有可按约束条数通用推导的界限；同样数量的断言可因理论、公式结构、变量耦合、量词、非线性及 SAT/UNSAT 查询方向而相差巨大。" 并建议比较三种方式（新 solver/一次性查询、assumptions、push/pop）后在本机实测选型。

本文强调（I）：这些是 "特定案例而非普遍基准"（相关研究 自己的措辞），因此不能被读作 "Z3 总是慢" 或 "push/pop 总是坏"。它们的作用是**否证** "按约束数估算延迟" 这一常见做法。这一区分——用具体案例否证普遍规律，而不是用它建立新规律——是文献综述中精确使用证据的范例。

7.4 并发：Context 隔离的官方依据

Z3 的对象归属 Context；官方 Python 并行示例说明并行工作应使用独立 context，且创建/访问共享 context 对象并发不安全（Z3 parallel.py；Context API）。相关研究 据此建议无共享 solver 的 worker pool，每个 worker 持有私有 Context/solver，任务间不跨 context 传递 AST，以可序列化 SMT-LIB 或项目 IR 在 worker 间传输。文档还警告："定期重建 solver。Z3 的增量能力存在，不等于所有查询都应 push/pop。"

本文认为（I）：这是把 "工具的内存模型" 翻译成 "服务的并发模型" 的一个干净实例，其推理方式可以复用到其他有状态工具（模型检查器、数据库连接）上。

性质/模型	判定状态与典型复杂度				

7.5 可判定性与复杂度：相关研究 的核心贡献

相关研究 第 3 章给出了本项目中最系统的一张复杂度表。其关键条目（F）：

性质/模型	判定状态与典型复杂度				
有限显式图可达性/SCC	‘O(	V	+	E	)‘；隐式编码状态可达指数级
有界模型检查（SAT/SMT BMC）	对深度 k 有限；unsat 只覆盖 k 步				
LTL 模型检查	模型与公式联合输入通常 PSPACE-complete（Schnoebelen）				
CTL/CTL* 模型检查	CTL 通常多项式；CTL* 更高，取决于编码与公式				
ATL/有限并发博弈	固定有限玩家有算法；参数化玩家数可显著变难甚至不可判定（ATL complexity）				

性质/模型	判定状态与典型复杂度				
概率 Markov/MDP/随机博弈	有限模型上许多 reachability/概率阈值问题可判定；并发/定量奖励/时间扩展复杂度显著上升				
普通 Petri 网 reachability	可判定，但已知**非 elementary** 复杂度下界（Czerwinski et al.）				
Presburger 算术（带量词）	可判定，但量词消去最坏复杂度至少超指数/可达三重指数（Survival Guide）				
非线性整数算术	一般**不可判定**（可编码 Hilbert 第十问题）（Undecidability and hardness in MINLP）				
无界通用程序/递归规则系统	一般 reachability/termination **不可判定**（停机问题）				

相关研究 最重要的一句方法论判断是：

“可判定性不等于可计算性价比；不可判定性也不等于没有实用算法。”

它以 Petri 网 reachability 为例：有算法但复杂度极高（非 elementary）；以非线性整数问题为

例：可判定的核心不存在，但可在有限边界上做 SAT/SMT 搜索；以无界规则为例：可通过抽象解释、ranking function 或 CHC 取得局部证明。**本文认为（I）这一判断是对 “可判定性” 最常见的两种误读（“可判定=可行” 与 “不可判定=无用”）的精确纠正，也是整份 相关研究 中最具理论价值的单句。**

7.6 分层验证与 CEGAR

相关研究 提出的六层算法层级（结构 → 抽象数值 → 离散动态 → 约束精化 → 无界证明 → 随机/博弈）以及层间的 CEGAR 循环，是该章的方法论核心（I）：

> 便宜过近似发现风险或筛选候选；反例回放确认是否真实；真实反例直接回译为规则路径；假反例提取谓词、版本、时间或玩家观察条件并精化。

本文的批判性分析（I）：这一设计与 相关研究 的 “区间传播作为全量快筛，SMT 用于疑难局部” 是同一思路的扩展。其理论工具（抽象解释的 Galois connection、过近似/欠近似、refinement）来自 Cousot & Cousot 1977%20-%20Abstract%20Interpretation.pdf) 与 CEGAR 综述。**但 相关研究 真正的贡献不是 “使用 CEGAR”，而是指出精化谓词的来源必须是游戏语义**（版本、时间、玩家观察、单位、缺省、例外）——这使它从 “应用已有技术” 变成 “定义一个新问题：如何从游戏语义自动生成精化谓词”。

相关研究 同时给出了一个重要的保守编译三分类（I）：**证明保持的过近似**（适合安全证明，产生假阳性）、**反例保持的欠近似**（适合找可执行漏洞，找不到不代表安全）、**未决定/待澄清**（规格缺少概率/单位/缺省/优先级/例外语义时不强行解释）。并列出必须防止的失败模式：开放世界的 “未提及” 被错译为 “禁止”；默认值被错译为常量；“每回合” 被错译为每秒；概率掉落被替换为期望值；例外规则排序丢失；版本覆盖被合并成同时生效；以及把策划希望（should）当成运行时强制条件（must）。

本文认为（I）：这份失败模式清单是 相关研究 最具实践价值的部分，因为它把 “语义鸿沟” 从一个抽象概念转化为七个可检查的具体错误类型，每一个都可以成为 benchmark 的实例类别。

7.7 正确构造 vs 事后验证

相关研究 明确界定了 correct-by-construction 的收益与边界：

收益：one-hot、typed transition、唯一主状态等约束可以构造性地排除结构非法状态，使验证器不必反复证明明显的结构事实，错误尽早反馈，后续模型规模更小。

边界（F/I）：one-hot 只证明同一时刻至多/恰有一个枚举状态，不证明每个状态可达；类型正确不证明数值边界、经济可持续或公平；迁移 API 强制前置资源不为空，不证明总存在可推进的下一步；版本 schema 合法不证明跨版本迁移保留玩家资产；随机分布格式合法不证明长期期望、

尾部概率和可玩性。

本文补充（I）：这一界定与 相关研究 的 “schema 合规 ≠ 事实正确” 在结构上完全同构——都是 “形式合法不等于语义正确”。本文认为这是整个项目最重要的跨层不变式：**每一层的形式保证都只覆盖该层，且都不覆盖下一层的语义**。

7.8 五个创新候选的评价

相关研究 提出五个候选，均标注为 “可检验候选，不是已完成的 novelty claim”。本文按研究价值与可证伪性排序评价：

最有潜力：候选一（带证据与不确定性标签的规则保守编译器）。理由是它把 相关研究 识别的最高风险（语义编译）转为一个可形式化的研究问题，并要求定义 “规则编译保守性” 及至少一个可判定子语言的编译/证明算法。它同时有明确的验证方式（构造含歧义/缺省/覆盖/例外的规则基准，度量漏报率、假反例率、真实反例回放率、来源可追踪率、版本回归保持率）。

其次：候选三（版本演化的精化保持与反例迁移）。理由是它对应真实的 live-service 需求，且与经典 refinement/回归验证有清晰的理论接口（双模拟/精化、度量精化、增量模型检查、反例映射）。**本文的批判（I）：**其难点在于 “兼容” 的定义——资源价值不下降、目标仍可达、旧存档可继续，是三种不同的性质，而 相关研究 已经指出这一点，因此该候选的第一个理论任务其实是**定义版本兼容性的偏序或等价关系**。

再次：候选二（守恒—可达—策略分层不变量综合）。理由是它区分了 “存在资源流”“存在合法操作序列”“玩家可长期维持” 三种性质，这正是第 3.3 节经济维度分析的自然延伸。相关研究 自己判断这是 “改进型”——组合成熟工具，只有在给出新不变量充分条件时才成为硬贡献。

候选四（玩家选择+随机+可玩性代理的定量反例）与候选五（反例/证明双向编译与求解器无关基准） 被 相关研究 标为 “改进型”。本文同意，并补充（I）：候选四的最大风险是 “可玩性” 无法被形式证明（相关研究 自己承认 “只能验证明确代理指标；对 fun、engagement 等属性需用户研究或仿真”），因此它必须把创新点严格限制在 “游戏语义模板 + 抽象保持定理 + 可扩展算法”，而不能声称验证了趣味性。候选五的价值取决于是否能产生社区复用的 benchmark；相关研究 正确指出 “单纯工程封装是弱创新”。

7.9 本章小结

形式化验证谱系是四个谱系中理论最成熟、也最容易被误用的一支。核心的三条认识是：**(1) 求解器只对被正确编码的模型负责**，因此自然语言编译正确性是最高风险环节；**(2) 可判定性不等于可计算性价比，不可判定性也不等于没有实用算法**，因此必须同时报告语法片段、边界与编码；**(3) 不同性质分属不同的量词结构**，不能统一压成 “是否存在死锁”。相关研究 提供了三类检查的具体编码，相关研究 把它们提升为一个 “带证据与版本的保守编译” 研究纲领。

8 横向比较与关键争议

8.1 四个谱系的接口：三个 “必须对齐” 的语义

把四个谱系并置后，可以看到它们之间存在三**个必须显式对齐的语义接口**。这三点是本综述的核心综合（I）：

接口一：约束强度与召回（谱系 B ↔ 谱系 A）。本体越严格，抽取越可审计，但越可能把 “本体未覆盖的真实事实” 判为无事实。相关研究 的 “结构正确地乱抽” 与 相关研究 的 “开放 `Concept` 会成为未知类型的兜底” 是同一张硬币的两面。争议点在于：**`uncovered` 应被视为模型失败还是本体缺口？** 上游的一致答案是 “两者都可能”，因此不能简单地把 `uncovered` 增多当作模型差（相关研究 明确列出这一风险）。

接口二：证据粒度与验证粒度（谱系 B ↔ 谱系 D）。抽取层以 chunk 为界，验证层以规则系统为界。相关研究 要求 “每条约束保留回映射（SMT assertion ↔ 图边/原文出处）”，相关研究 要求 “每个 SMT/SAT/CHC assertion 可回指规则 claim 和原始证据”。**这意味着证据绑定必须贯穿到求解器的 assertion 级别**，而不能停在图谱边级别。本文认为这是 LudoGraph 相对通用 KGC 工作的一个结构性差异，也是一个可发表的技术要求。

接口三：不确定性与证明边界（谱系 B ↔ 谱系 D）。相关研究 的四态 `certainty(explicit/possible/planned/neg` 与 相关研究 的六类结果协议必须能组合。争议点在于：**一条 `possible` 的 claim 能否进入形式化模型？** 相关研究 的回答是 “语义不明的 claim 不可强行编码，应返回’ 未建模/待确认’”；相关研究 的回答是引入 “未决定/待澄清” 作为第三种编译结果。两者的立场一致：**不确定性不能被求解器吸收，必须外化**。

8.2 三组实质性争议

争议一：端到端还是分层？ 相关研究 观察到 2023–2026 的主趋势是分层组合而非端到端取代。但 相关研究 明确指出，“两步路由优于端到端” 在 Text2Cypher 上**缺少对等实验支持**——公开基准测的是端到端准确率。本文的判断（I）：这不是矛盾，而是**证据强度的差异**。相关研究 的 “分层趋势” 是**描述性观察**（多个系统的共同结构），相关研究 的 “两步更好” 是**因果性主张**（缺少实验）。前者可以作为研究方向的依据，后者必须被标注为待验证假设。这一区分应被写进未来论文的 related work。

争议二：LLM-as-a-judge 能否作为评测证据？ 相关研究 记录了 KARMA、MINE-1/Wikontic 等使用 LLM 裁判的结果（如 KARMA 报告 LLM 验证正确率 83.1%、冲突边下降 18.6%；Wikontic 报告 MINE-1 信息保留率约 84–86%）。同时 相关研究 自己指出这类结果 “可能偏好流畅而非忠实输出；需要人工盲评和 judge 互换/校准”。**本文的判断（I）：**这与 相关研究 的 “证据蕴含难判定” 是同一个认识论问题——**用模型评估模型的输出，在方法上是一个未闭合的循环**。因此任何仅以 LLM 裁判为证据的结论，在本文中都只能作为 “弱证据” 处理，不应进入关键论证。这也是 LudoGraph 自建 gold set 的必要性来源。

争议三：“毫秒级” 是否可承诺？ 预调研报告的摘要在描述目标时使用了 “高并发、毫秒级静态形式化验证”。但 相关研究、相关研究、相关研究、相关研究 全部给出了否证性证据：

- 相关研究：公开基准不构成跨产品可比的写入峰值或毫秒 SLA。
- 相关研究：总延迟 = 路由 + schema/示例检索 + 查询编译 + 数据库执行 + 结果整理，“毫秒级” 只适用于缓存、规则、模板和已编译查询这类快路径。
- 相关研究：SMT 的运行时间没有可按约束条数通用推导的界限；“毫秒级” 仅承诺给固定模板、受限理论、有界小子图的热路径。
- 相关研究：所有 “毫秒级”“线性扩展” 表述在项目中均应写成待测假设。

本文的立场（I）：这不是四份文档与预调研报告矛盾，而是**四份后续调研对一份早期蓝图的目标表述进行了精确化**。文献综述的正确处理方式是把 “毫秒级” 重新表述为**分层、分场景、待实测的延迟目标**，而不是删除或保留其模糊形式。这是 5C 中 “一致性” 原则的直接应用。

8.3 一个一致的方法论倾向

跨四章可以观察到上游文档共享一种可辨识的方法论倾向，本文将其命名为**“保守主义的三重表现”（I）**：

1. **抽取保守：**宁可少抽，明确暴露 `uncovered`，也不把模型常识伪装成图谱事实（相关研究）。
2. **存储保守：**冲突并存不覆盖，修订追加不删除，投影不取代规范断言历史（相关研究）。
3. **验证保守：**不把有限深度 UNSAT 当全局安全，不把 `unknown` 当通过，不把求解器结论当作现实结论（相关研究、相关研究）。

本文认为（I）这三者不是三个独立的设计选择，而是同一个认识论立场在三个层次上的投射：系统承认自身知识的边界，并把边界外化为可审计的状态。这是 LudoGraph 相对通用 KGC 系统最可能形成方法论贡献的地方，因为它同时拒绝了两个极端——既不像纯 LLM 系统那样把不确定性隐式消化，也不像传统形式化项目那样要求输入已被手工形式化。

8.4 文档间的时序与前提不一致

作为批判性综述，有必要记录上游文档之间的四处不一致（I，均为前提演进而非错误）：

1. **Kùzu 前提失效。**相关研究 以“保留 Kùzu 的存储选项”为前提分析方言兼容性，相关研究 发现 Kùzu 仓库已归档。
2. **“毫秒级”的精确化。**见 7.2 争议三。
3. **验证抽象层级。**相关研究 的三类检查在 相关研究 中被重新组织为六层方法论，`unknown` 的表达也从三类扩展为六类结果协议。
4. **本体计数。**相关研究 记录了设计文档的 25/24 关系计数不一致（F）。

本文的处理方式是**保留每份文档在其自身日期下的有效性，同时标注跨文档的可组合性前提**，而不是把它们统一改写为“最新版本”。

9 Research Gap

9.1 空白的形式化表述

基于前七章的分析，本文把 LudoGraph 面对的研究空白表述为一个**联合缺口**，而非单一缺口（I）：

> **缺口：**现有文献分别提供了（a）分析导向的游戏本体理论、（b）受本体约束的抽取与证据绑定技术、（c）LPG/查询编译工程、（d）形式化验证理论，但**没有一个公开、可复现的工作把“自然语言游戏规则文本 → 带连续证据与版本的本体化 claim → 可判定验证问题 → 可回译的反例/**

> **证明" 作为一条端到端管线来定义、实现和评测**。更具体地说，缺少的不是任一组件，而是**跨组件的语义契约**：claim 的限定字段如何映射到 IR 的量词/时间/概率语义，抽取的 `certainty` 如何映射到验证的结果协议，版本覆盖如何处理为可证明的精化关系。

相关研究 第 6 章的检索结论（"未发现一个已被广泛采用、公开可复现、专门从商业游戏策划文档/规则文本构建本体化 KG 的 LLM benchmark 或成熟系统"）被显式标注 `[unverified]`。本文沿用这一谨慎。**但即使这一否证不成立**，缺口的形式化表述仍然成立：因为它的核心是**跨组件语义契约的缺失**，而不是某个具体系统的不存在。

9.2 六类可检验的具体空白

本文把整体缺口分解为六类可操作、可证伪的具体空白：

G1：条件/模态/否定/时序的作用域抽取与评测。现有 benchmark（Text2KGBench 的短句与固定 ontology）不覆盖 "当 X 且非 Y 时每回合一次地 Z" 这类作用域结构。相关研究 明确指出 "`if/unless/only while/once per turn/at reset` 的作用域和规则方向是否正确；目前通用 benchmark 覆盖不足"。

G2：数值与单位保真。raw value、范围、单位、周期、概率及舍入；防止模型把 "每局" 换算成 "每分钟"。相关研究 的 validator 设计要求 "保存 `raw`、unit、范围和 qualifier；只有明确可解析时才填归一化 value，禁止隐式换算"。

G3：版本与冲突。同一规则在不同版本、模式或来源冲突时并存、标注来源而不被错误融合。相关研究 候选三（版本演化的精化保持与反例迁移）直接对应这一空白。

G4：保守拒绝能力的度量。unknown/uncovered 的 precision、recall、拒绝错误关系的比例。相关研究 明确指出 "不能只奖励召回"。

G5：抽取到形式化的保守编译。定义 "规则编译保守性"，给出过近似/欠近似/未决定的编译契约，并证明其在受限子语言上成立。这是 相关研究 候选一的核心，也是 相关研究 识别的 "最高风险环节"。

G6：反例回译与求解器无关基准。让 SAT/SMT/Petri/CHC/显式模型检查的结果都能回译为策划可读的规则路径、库存变化、版本和来源，并建立按理论片段、状态规模和语义维度分层的公开基准。这是 相关研究 候选五。

9.3 无法回避的 "否证不完整" 问题

在给出上述空白后，必须诚实指出一个方法学限制（U）：**所有 "G1–G6 未被覆盖" 的判断都依赖公开检索，而检索的否证是不完全的**。相关研究 明确指出 "较小众会议、工业内部项目、未索引仓库和 2026 新论文可能遗漏"。因此本文的建议是：**在把这六类空白写成论文贡献之前，必须按 相关研究 第 7.2 节的建议做第二轮系统综述与专利排查**（关键词组合如 `game rules + conservative compilation`、

versioned stochastic rule verification、playability formal verification)。在完成该排查前，六类空白应被表述为**研究假设**，而非已确立的空白。

10 LudoGraph 的研究合法性与具体切入点

10.1 研究合法性论证

一个研究项目需要回答 “为什么这个问题值得研究且尚未被解决”。本文给出四条合法性论证（I）：

论证一：问题的三个层次真实存在且相互独立。抽取、表示、验证分别有独立的技术栈与失败模式，且没有任何一层能替代另一层（第 4.1、6.7 节）。这意味着该问题不能被简化为 “用 LLM 抽取三元组” 或 “用 Z3 求解约束” 的任一单项工作。

论证二：游戏领域确实落在现有基准的盲区上，且盲区有明确内容。第 8.2 节列出六类可检验的空白，每一类都可以指认现有工作的具体不覆盖点（短句、固定 ontology、社区摘要、期望值替代随机、固定 schema、QA 指标）。这使 “游戏领域有空白” 从直觉变为可指认的判断。

论证三：现有游戏领域的公开工作解决的是不同问题。相关研究 指出最接近的是 AriGraph：在交互式文本游戏中从观察和经历构建 episodic/semantic memory graph 并用于 agent 规划；但它 “不从规则文档抽取显式条件/数值/版本，也不以可审计本体为目标”。另一条先例是 VGDL 传统（把游戏规则表达成可执行语言，并以规则生成作为竞赛问题），它是 “形式化规则语言/可执行规则” 的证据，不是 LLM-KGC construction。**因此游戏领域的现有工作覆盖的是 “agent 记忆” 与 “可执行规则生成” 两端，而中间 “从策划文档到可审计本体化 KG 再到验证” 的一段是空的。**

论证四：架构方向与当前研究前沿一致而非逆流。相关研究 观察到的 “生成候选 → 检索/本体对齐 → 验证/融合 → 求解” 分层组合趋势，与非对称神经-符号架构在结构上一致。这意味着项目不需要说服审稿人接受一个逆行的方法论。

10.2 具体切入点（按优先级）

本文基于第 6.8 节对 相关研究 五个候选的评价，给出三个具体切入点（I）：

切入点 A：带证据与版本的规则保守编译器（最高优先级）。

- **问题定义：**将自然语言、配置表、版本覆盖和例外优先级编译为带来源映射的 IR，并同时输出过近似安全模型、欠近似反例模型及未决定项。
- **理论工具：**refinement relation、抽象解释的 over/under-approximation、三值/多值逻辑、规格精化、来源追踪、可验证编译器。
- **硬性成果形态：**形式化定义 “规则编译保守性” 及至少一个可判定子语言的编译/证明算法。
- **为什么最优：**它同时覆盖 G1（条件/时序作用域）、G2（数值保真）、G3（版本）、G5（保守编

译）四类空白，且 相关研究 已独立指认它是最高风险环节。

切入点 B：版本演化的精化保持与反例迁移（次高优先级）。

- **问题定义：**当掉落率、配方、状态枚举或例外规则改变时，判断新旧版本是否保持既有安全性、可达性和玩家资产迁移性质，并把旧版本反例自动迁移或解释为已失效。
- **第一个理论任务：**定义 “版本兼容” 的偏序或等价关系（资源价值不下降 / 目标仍可达 / 旧存档可继续是三种不同性质）。
- **理论工具：**双模拟/精化、迁移系统关系、概率/加权系统的度量精化、版本化 IR、增量模型检查、反例映射。
- **为什么优选：**它直接对应 live-service 的真实需求，且与经典 refinement 有清晰的理论接口，容易产生可证明的定理而非仅工程实现。

切入点 C：守恒—可达—策略分层不变量综合（第三优先级）。

- **问题定义：**区分 “存在一个资源流”“存在合法操作序列”“玩家在对抗环境下能持续维持库存” 三种性质，自动生成线性不变量、势函数和必要的 Petri/博弈精化查询。
- **理论工具：**place/transition invariants（Basic linear algebraic techniques for Petri nets）、LP/ILP、ranking/potential functions（The Automatic Synthesis of Linear Ranking Functions）、MDP/ATL、CHC 归纳不变量。
- **为什么第三：**相关研究 自己判断这是 “改进型”；只有在给出针对游戏 “净产出 + 可玩行动下界 + 对手/随机量词” 的新不变量充分条件时，才成为较硬的理论贡献。

10.3 一个被低估的切入点：抽取—验证接口的负结果研究

本文提出一个上游未明确列出、但由第 7.1 节接口分析自然导出的切入点（I）：

切入点 D：抽取不确定性与验证结论之间的可组合性研究。

核心问题是：当图谱中一条关键 claim 的 `certainty` 是 `possible`、或存在两个冲突来源、或某个限定字段缺失时，验证结论应如何降级？ 可能的输出不是 “pass/fail”，而是 “在假设 A 下 proved、在假设 B 下 counterexample、在假设 C 下 not-modeled”。这需要：

- 定义 `certainty` × 编译结果 的**组合语义表**；
- 定义冲突 claim 的**分支验证**（对每个来源分别编译并比较结论）；
- 定义缺失字段的**保守默认**（缺省时取过近似还是拒绝编译）。

为什么这可能是好的研究：它把一个通常被当作工程细节的问题提升为语义问题，并且其结论可以独立于任何具体求解器发布（一张语义表 + 一组定理）。同时它天然地产生 benchmark（含

歧义、冲突、缺省的规则集），从而也覆盖 G4（保守拒绝度量）与 G6（求解器无关基准）。

10.4 避免的伪贡献

作为批判性综述，也需要指出哪些方向**不应**作为主要贡献（I）：

- **“用 LLM 抽取游戏三元组” 本身**。这与 2023 年以来的通用 KGC 工作无本质区别，且无法回答 G1–G6。
- **“把配置字段转换成 SMT”**。相关研究 明确判断 “仅把配置字段转换成 SMT 是应用型工作”。
- **“用 agent 数量证明可靠性”**。相关研究 指出 “agent 数量本身不是可靠性指标；只有独立证据、确定性 validator 或 solver-backed ablation 才能证明增益”。
- **“毫秒级端到端验证” 的工程封装**。第 7.2 节争议三已说明该指标缺少可外推的公开证据。
- **“可玩性/趣味性的形式化证明”**。相关研究 明确：“只能验证明确代理指标；对 fun、engagement 等属性需用户研究或仿真，不能冒充形式证明。”

11 风险与未知

11.1 按严重程度分级

本文按 “若发生则使研究结论无效” 的程度排序（I）：

R1（最高）：自然语言编译的正确性。相关研究 明确指认 “最高风险不是 Z3 API 本身，而是从知识图谱 claim 到形式化语义的正确、保守编译”。若 “通常”“可选”“每周”“保底”“优先级” 被错误形式化，后端可以对错误模型给出完全正确的证明。相关研究 补充：开放世界/闭世界差异、缺失事实、单位、时间/版本及并发更新均可能让模型与游戏实际行为不一致。**这个风险贯穿抽取、本体、IR 和验证结果解释，不能由换求解器解决**。

R2（高）：“经济可持续” 与 “可玩性” 的定义随产品变化。相关研究 指出：玩家市场、交易、赛季重置和外部付费会让封闭资源模型失效；未定义系统边界时，任何全局稳定性结论都应标为 `[unverified]`。“有解” 不等于 “有意义的选择”，概率成功不等于好玩。

R3（高）：公开游戏案例不足以证明大型系统可扩展性。相关研究 明确：“现有视频游戏 Petri 网和 Promela 研究支持可行性先例，但尚未找到覆盖大型商业经济、版本演化、概率尾部与高并发玩家策略的统一公开实测。” 该 “未找到” 标为 `[unverified]`。

R4（中高）：本体覆盖不足的双向风险。过严的本体会把 “本体缺口” 误判为 “文本无事实”（相关研究、相关研究）；过松的本体（如开放 `Concept`）会让未知类型逃避判定，降低可审计性（相关研究）。**这不是可以通过调参解决的问题，而是需要持续的本体演化治理**。

R5（中）：证据蕴含的不可判定性。相关研究 指出：“quote 连续且相关，不代表它支持 claim

的全部方向、条件和数量；需要规则 + NLI/人工复核，且 NLI 本身可能幻觉。” 这意味着证据绑定解决了 “有无原文” 的问题，但没有解决 “原文是否支持该 claim” 的问题。

R6（中）：LLM-as-a-judge 的评估偏差。相关研究 指出 KARMA、MINE-1/Wikontic 等结果 “可能偏好流畅而非忠实输出；需要人工盲评和 judge 互换/校准”。

R7（中）：求解器性能结论不可外推。相关研究 与 相关研究 一致：当前公开资料没有给出 LudoGraph 真实分布下约束数、状态数、抽象域与 P95/P99 的曲线。Z3 的具体性能案例（0.03 秒 → 151.32 秒）是特定案例而非普遍基准。

R8（中）：多 agent 的可靠性与成本。相关研究：“多 agent 可能提高审查覆盖，也可能重复调用、相互确认错误、泄露上下文或死循环。”

R9（中）：存储层的一致性风险。相关研究 指出 Assertion 节点增添遍历跳数，会影响热点查询；物化投影带来双写/一致性与删除重建风险；且 “namespace 字段本身不是安全边界”。

R10（低中）：许可证与数据合规。相关研究 指出 Neo4j Community GPLv3 的使用、修改、分发和网络服务适用边界需由法务按实际交付方式确认；相关研究 指出 Text2KGBench 的上游 TekGen/WebNLG 有各自条款，“论文仓库的代码许可不自动覆盖数据、模型权重和商业输入”。

11.2 明确标注的未知项

以下项在上游被标注 `[unverified]` 或明确为未决，本文原样保留：

编号	未知项	来源
U1	是否存在公开、成熟、专门针对游戏策划文档的 ontology-constrained extraction benchmark	相关研究
U2	是否存在专门从商业游戏策划文档构建本体化 KG 的成熟系统（强否证版本）	相关研究
U3	国内是否有同等成熟的中文游戏本体研究（未检索 CNKI/万方等）	相关研究、相关研究
U4	2026 年新论文、引用量、仓库活跃度的稳定索引	相关研究

编号	未知项	来源
U5	同一 NL2Cypher 数据集上 FP16 vs INT4、同一端侧设备的完整可复现对比	相关研究
U6	LudoGraph 3B LoRA 的具体显存/时长	相关研究
U7	不同 provider 对复杂 JSON Schema、流式输出、refusal 和 schema 子集的支持一致性	相关研究
U8	游戏经济 Z3 约束数—延迟曲线或端到端高并发实测	相关研究
U9	游戏规则版本演化语义契约的公开覆盖情况	相关研究
U10	覆盖随机、时间、版本、默认和玩家策略的游戏规则验证统一 benchmark	相关研究
U11	FalkorDB、GraphDB 的具体许可与商业报价	相关研究
U12	五个创新候选的 novelty（需第二轮专利/论文检索）	相关研究
U13	17 标签覆盖率、新增维度的准确率提升、关系拆分降低歧义等量化证据	相关研究

11.3 不确定性如何影响本文的结论

必须诚实说明：本文第 8 章的六类空白与第 9 章的三至四个切入点，其 “未被覆盖” 的前提部分依赖 U1、U2、U3、U12。若第二轮系统综述发现已有工作覆盖其中某类，对应的切入点应被重新定位为 “改进与扩展” 而非 “填补空白”。**因此本文的所有空白主张都应被读作 “在当前检索证据下成立的假设”**。

另一方面，本文的核心分析（三个层次的分离、三个接口的对齐、保守编译作为最高风险、可判定性 ≠ 可计算性价比）**不依赖**上述否证——它们是对**已核实文献内容**的结构性综合，即使某个未发现的系统存在，这些分析仍然成立。

11.4 可证伪的验证计划

本文建议以下验证计划，作为把上述空白与切入点转化为可检验研究的最小工作集（I，采纳并扩展 相关研究 第 6.5 节的建议）：

最小可行研究集（每个样例需人工解释 + 形式 oracle + 最小反例）：

1. 资源环但无死锁（依赖环存在，但初始资源与顺序允许持续运行）；
2. 资源环且初始不可启动（真正的经济死锁）；
3. 可行流但不可执行（LP 有解，但中间库存非负约束被违反）；
4. one-hot 合法但目标不可达；
5. 概率均值安全但尾部失控（期望产出可接受，但极端连续失败/成功越界）；
6. 版本变更破坏旧存档（掉落率或配方改变导致既有玩家资产性质失效）；
7. 例外优先级冲突（两条规则同时适用且顺序不同结论不同）；
8. 玩家存在/不存在强制策略（∃ 策略 ∀ 环境 的量词检验）。

评测协议（两层分离）：

- **抽取层：**edge/claim precision-recall、证据覆盖率（`quote == chunk[start:end]` 应为 100%，硬门槛）、unknown 拒绝 precision/recall、domain/range 违反率（目标 0）、实体重复率、图级冲突率、`certainty` 四态 macro-F1、多次采样一致率、P50/P95 延迟与 token 成本。
- **验证层：**solver 可编译率、反例可回溯率（SMT assertion → 图边 → 原文 span）、`proved` / `counterexample` / `bounded-no-counterexample` / `unknown` / `not-modeled` 的分类统计、假反例率、真实反例回放成功率、按理论片段（QF_LIA / QF_BV / CHC / 随机博弈）分层的 P50/P95/P99 与内存。

关键要求：报告必须分别给出抽取层与验证层的指标，不能只给最终 QA 分数（相关研究 第

10.4 节）；必须报告未知与恒等映射（相关研究、相关研究）；必须使用固定 commit、模型版本、schema 版本与 benchmark 配置（相关研究 第 10.6 节）。

12 结论

12.1 对文献的判断

本综述对 既有专题调研 覆盖的学术谱系给出如下判断：

成熟的部分。游戏设计理论提供了足够的分层语汇（MDA/Sicart/GOP/GDL/Temporal Frames）；约束解码与结构化输出已产品化；本体校验的工具链完整；属性图数据库与 Cypher 生态成熟；Petri 网、模型检查、抽象解释、SMT/CHC、随机博弈各自有稳固理论。在**组件层面**，LudoGraph 不需要发明任何基础技术。

不成熟的部分。跨组件的语义契约缺失。具体表现为：本体如何在 “分析可用” 与 “可编译” 之间取得平衡；抽取的 `certainty` 如何与验证的结果协议组合；证据绑定如何贯穿到 solver assertion 级别；时间、版本、概率、例外如何被一致地表达为可判定约束；以及 “格式正确 / 事实正确 / 模型正确” 三层次的分离如何在系统层面被强制。

空白的形态。游戏领域的空白不是 “没人做过游戏 KG”，而是 “没人把策划文档到可验证规则的完整语义链作为研究对象”。这一空白的强否证版本标为 `[unverified]`，但其核心（跨组件契约缺失）不依赖否证。

12.2 对 LudoGraph 的判断

研究合法性成立，但需要精确化目标表述。项目的四层结构（文本抽取 → 领域本体 → 规则/数值/时序 IR → 符号验证）与当前研究前沿的分层组合趋势一致。但预调研报告中 “高并发、毫秒级” 的表述必须被改写为分层、分场景、待实测的延迟目标，因为四份后续调研都给出了否证性证据。

推荐的研究主线。从 “求解器接入” 转向 “语义与证明边界”：形式化定义**规则编译保守性**，给出过近似/欠近似/未决定的三分编译契约，在至少一个可判定子语言上证明其 soundness，并让版本、概率、时间、例外与证据冲突成为一等语义。Z3、SPIN、TLC、PRISM、Petri 工具只是后端。

推荐的切入点顺序。A（带证据与版本的规则保守编译器）> B（版本演化的精化保持与反例迁移）> C（守恒—可达—策略分层不变量综合），并把 D（抽取不确定性与验证结论的可组合性）作为一个可独立发布的方法论副产品。

推荐避免的方向。通用三元组抽取、配置到 SMT 的机械转换、以 agent 数量替代可靠性证据、“毫秒级” 工程封装、“可玩性” 的形式化证明。

12.3 最终的元判断

本文认为，LudoGraph 最有价值的研究贡献可能不是某个新算法，而是一**套“保守性契约”**：规定从自然语言到形式模型、从抽取到验证、从版本到版本的每一个转换中，什么被保持、什么被丢弃、什么被显式标记为未决定。这类契约在软件验证（refinement obligation）、编译器（verified compilation）、需求工程（trustworthy formal NL specifications）中都有先例，但**尚未被应用到游戏规则这一特定领域**（I）。

如果这一判断成立，那么该项目的论文贡献形态将不是“我们做了一个系统”，而是“我们定义了游戏规则编译保守性，在受限子语言上证明了它，并用可证伪的 benchmark 展示了现有方法的失效模式”。这是一个更小、更硬、也更可审稿的贡献。

13 参考文献与可核验链接

以下链接均来自 既有专题调研 与预调研报告所引用的来源。标注 `[unverified]` 的条目表示上游未能固定其作者、许可证或版本，或属于检索否证。

13.1 游戏理论与本体

1. Hunicke, LeBlanc & Zubek, *MDA: A Formal Approach to Game Design and Game Research* — https://www.cs.northwestern.edu/~hunicke/MDA.pdf；AAAI 页面 — https://aaai.org/papers/ws04-04-001-mda-a-formal-approach-to-game-design-and-game-research/
2. Sicart, *Defining Game Mechanics* — https://gamestudies.org/0802/articles/sicart
3. Game Ontology Project (GOP) 官网 — https://gameontology.org/
4. Junior & Silva, *RMDA* 游戏设计本体（2021，提议性研究）— https://www.it.pt/Publications/DownloadPaperJournal/31734
5. Stanford GDL Overview — http://ggp.stanford.edu/overview/game_description.html
6. Stanford GDL Specification — http://ggp.stanford.edu/readings/gdl_spec.pdf
7. GDL-III（IJCAI 2017）— https://www.ijcai.org/proceedings/2017/177
8. Zagal & Mateas, *A Unifying Framework for the Analysis of Game Temporality* — https://users.soe.ucsc.edu/~michaeln lications/zagal-digra2007.pdf
9. Mitchell et al., *Towards an Agency-centered Ontology of Game Mechanics* — https://dl.acm.org/doi/10.1145/3582437
10. *A Practical Guide Into Creating a F2P Game Economy* — https://www.gamedeveloper.com/design/a-practical-guide-into-creating-a-f2p-game-economy—a-cake-recipe
11. Unity State Machine Basics — https://docs.unity3d.com/6000.6/Documentation/Manual/StateMa-

chineBasics.html

12. *A Holistic Look at Requirements Engineering in the Gaming Industry* — https://arxiv.org/html/1811.03482v1
13. *How practitioners approach gameplay requirements?* — https://doi.org/10.1007/s10664-019-09711-4

13.2 神经符号抽取与知识图谱构建

1. OpenAI, *Introducing Structured Outputs in the API* — https://openai.com/index/introducing-structured-outputs-in-the-api/
2. OpenAI, *Function calling* — https://platform.openai.com/docs/guides/function-calling
3. Anthropic, *Tool use* — https://docs.anthropic.com/en/docs/build-with-claude/tool-use
4. Google, *Structured output* — https://ai.google.dev/gemini-api/docs/structured-output
5. Outlines — https://github.com/dottxt-ai/outlines
6. Guidance — https://github.com/guidance-ai/guidance
7. LM Format Enforcer — https://github.com/noamgat/lm-format-enforcer
8. llama.cpp grammars — https://github.com/ggerganov/llama.cpp/blob/master/grammars/README.md
9. Wei et al., *Chain-of-Thought Prompting* — https://arxiv.org/abs/2201.11903
10. Wang et al., *Self-Consistency Improves Chain of Thought Reasoning* — https://arxiv.org/abs/2203.11171
11. Turpin et al., *Language Models Don't Always Say What They Think* — https://arxiv.org/abs/2305.04388
12. CoT 综述 — https://arxiv.org/abs/2309.15402
13. Garcez & Lamb, *Neural-Symbolic Learning and Reasoning: A Survey* — https://arxiv.org/abs/2109.05308
14. FEVER — https://fever.ai/
15. ERASER benchmark — https://www.eraserbenchmark.com/
16. Evidence Inference — https://github.com/jayded/medical-evidence-inference
17. DocRED — https://github.com/thunlp/DocRED
18. SciERC — https://github.com/allenai/SciERC
19. TACRED — https://nlp.stanford.edu/projects/tacred/
20. UIE — https://github.com/universal-ie/UIE
21. REBEL — https://github.com/Babelscape/rebel
22. LLMs4OL — https://github.com/HKUST-KnowComp/LLMs4OL
23. JSON Schema — https://json-schema.org/
24. Text2KGBench — https://arxiv.org/abs/2308.02357；仓库 — https://github.com/cenguix/Text2KG-

Bench

25. *A Comprehensive Survey on Automatic Knowledge Graph Construction* (ACM CSUR) — https://dl.acm.org/doi/10.11
26. *LLM-empowered Knowledge Graph Construction: A Survey* — https://arxiv.org/abs/2510.20345
27. AdaKGC（Schema-adaptable KGC）— https://arxiv.org/abs/2305.08703
28. ChatIE — https://arxiv.org/abs/2302.10205
29. EDC（Extract, Define, Canonicalize）— https://aclanthology.org/2024.emnlp-main.548/；代码 — https://github.com/clear-nus/edc
30. *Ontology-guided KG Construction from Maintenance Short Texts*（KALLM 2024）— https://aclanthology.org/2024.kallm-1.8.pdf
31. GraphRAG — https://arxiv.org/abs/2404.16130；仓库 — https://github.com/microsoft/graphrag
32. HippoRAG — https://proceedings.neurips.cc/paper_files/paper/2024/file/6ddc001d07ca4f319af96a3024f6dbd1-Paper-Conference.pdf；仓库 — https://github.com/osu-nlp-group/hipporag
33. LightRAG — https://arxiv.org/abs/2410.05779；仓库 — https://github.com/HKUDS/LightRAG
34. KGGen / MINE-1 — https://arxiv.org/abs/2502.09956
35. KARMA（Multi-Agent Knowledge Graph Enrichment）— https://arxiv.org/abs/2502.06472
36. Graphusion — https://arxiv.org/abs/2407.10794
37. AriGraph（IJCAI 2025）— https://www.ijcai.org/proceedings/2025/2
38. AGENTiGraph — https://arxiv.org/abs/2508.02999；仓库 — https://github.com/SinketsuZao/AGENTiGraph
39. Wikontic — https://arxiv.org/abs/2512.00590；仓库 — https://github.com/screemix/Wikontic
40. SymAgent — https://arxiv.org/abs/2502.03283
41. Logic Query of Thoughts (LQOT) — https://arxiv.org/abs/2404.04264
42. KG-Prover — https://arxiv.org/html/2503.11657
43. Dialogue Ontology Relation Extraction via Constrained CoT Decoding (SIGDIAL 2024) — https://aclanthology.org/anthology-files/anthology-files/pdf/sigdial/2024.sigdial-1.33.pdf
44. PARSE（EMNLP Industry 2025）— https://aclanthology.org/2025.emnlp-industry.184.pdf
45. ExtractBench — https://github.com/Team-TTT/ExtractBench `[unverified]`

13.3 图存储与查询路由

1. Neo4j 开源/许可说明 — https://neo4j.com/open-source-project/ ；open-core FAQ — https://neo4j.com/open-core-and-neo4j/
2. Neo4j Cypher 关系模式参考 — https://neo4j.com/docs/cypher-manual/current/patterns/reference/node-and-relationship-patterns/
3. Neo4j 约束 — https://neo4j.com/docs/cypher-manual/current/schema/constraints/
4. Kùzu 仓库（归档状态） — https://github.com/kuzudb/kuzu ；Cypher 手册 — https://kuzudb.github.io/docs/cypher/ ；数据类型 — https://kuzudb.github.io/docs/cypher/data-types/
5. LadybugDB 仓库 — https://github.com/LadybugDB/ladybug ；文档 — https://docs.ladybugdb.com/
6. Memgraph 仓库及许可 — https://github.com/memgraph/memgraph ；BSL 许可文件 — https://github.com/memgraph/memgraph/blob/master/licenses/BSL.txt
7. FalkorDB 文档 — https://docs.falkordb.com/ ； 仓库 — https://github.com/FalkorDB/FalkorDB
8. Apache AGE — https://age.apache.org/
9. Oxigraph 仓库 — https://github.com/oxigraph/oxigraph ；SPARQL 能力说明 — https://github.com/oxigraph/oxigraph/wiki/SPARQL
10. Ontotext GraphDB 产品页 — https://www.ontotext.com/products/graphdb/
11. Kùzu 与 Neo4j 嵌入式对比研究 — https://github.com/prrao87/kuzudb-study ；说明文章 — https://thedataquarry.com/blog/embedded-db-2
12. *A Benchmark Dataset towards Text2Cypher Task*（EMNLP 2025 Industry） — https://aclanthology.org/2025.emnlp-industry.133.pdf
13. *Text2Cypher: Bridging Natural Language and Graph Databases* — https://aclanthology.org/2025.genaik-1.11/
14. *Refining Text2Cypher on Small Language Model with RL* — https://www.mdpi.com/2076-3417/15/15/8206
15. Neo4j JDBC Text2Cypher 官方手册 — https://neo4j.com/docs/jdbc-manual/current/text2cypher/
16. llama.cpp IQ4_NL CPU benchmark — https://github.com/ggml-org/llama.cpp/pull/10541
17. llama.cpp / vLLM 比较讨论 — https://github.com/ggml-org/llama.cpp/discussions/15180

13.4 形式化验证、可判定性与求解器

1. Cousot & Cousot, *Abstract Interpretation: A Unified Lattice Model*（1977）— https://wcl.cs.rpi.edu/pilots/library/papers/TAGGED/3936-Cousot_Cousot%20(1977)%20-%20Abstract%20Interpretation.pdf
2. *Automated Abstraction Refinement for Model Checking Large Systems*（CEGAR）— https://www.cs.cmu.edu/~emc/15820A/reading/fmcad-cegar.pdf
3. *Trustworthy Formal Natural Language Specifications*（Onward! 2023）— https://csgordon.github.io/publications/onward23/onward23.pdf
4. *Bridging the Gap between Natural Language and Formal Specifications* — https://ceur-ws.org/Vol-1564/paper20.pdf
5. *Refinement by interpretation in π-institutions* — https://webarchive.di.uminho.pt/haslab.uminho.pt/lsb/files/1106.409
6. *Detecting structural errors in scene-based Multiplayer Games* — https://www.kom.tu-darmstadt.de/papers/RGS15-1.pdf
7. *Formal approach based on Petri nets for modeling and verification of video games* — https://www.cai.sk/ojs/index.php/cai/article/download/2021_1_216/1081
8. Girard et al., *A survey of siphons in Petri nets* — https://cedric.cnam.fr/fichiers/art_3836.pdf
9. *Basic linear algebraic techniques for Petri nets* — https://www.cse.iitb.ac.in/~akshayss/courses/cs735/desel-alg.pdf
10. *Decidability Issues for Petri Nets* — https://tidsskrift.dk/brics/article/download/21662/19099/49254
11. Czerwinski et al., *The Reachability Problem for Petri Nets Is Not Elementary* — https://arxiv.org/abs/1809.07115
12. Schnoebelen, *The Complexity of Temporal Logic Model Checking* — https://lsv.ens-paris-saclay.fr/Publis/PAPERS/PDF/Sch-aiml02.pdf
13. Vardi, *Automata-Theoretic Verification* — https://www.cs.rice.edu/~vardi/papers/hba11.pdf
14. *Asynchronous Games on Petri Nets and ATL* — http://arxiv.org/pdf/2107.06866v1
15. *On the Expressiveness and Complexity of ATL* — https://lmcs.episciences.org/826
16. *Parameterized Model Checking for Alternating-time Temporal Logic* — https://www.doc.ic.ac.uk/~alessio/papers/16/ECAI16-KL.pdf
17. Kwiatkowska et al., *Quantitative Verification and Strategy Synthesis for Stochastic Games* — https://www.prismmodelchecker.org/papers/ejc-games.pdf
18. *PRISM-games*（TACAS 2013）— https://prismmodelchecker.org/papers/tacas13.pdf
19. *PRISM-games 3.0*（CAV 2020）— https://www.prismmodelchecker.org/papers/cav20pg3.pdf
20. *Timed Temporal Logics* — https://people.mpi-sws.org/~joel/publications/timed_temporal_logics17.pdf

21. Parker et al., *Verification of Probabilistic Real-time Systems* — https://www.cs.ox.ac.uk/David.Parker/papers/etr13.pdf
22. Z3, *Programming Z3* — https://theory.stanford.edu/~nikolaj/programmingz3.html
23. Z3 Python parallel contexts 示例 — https://github.com/Z3Prover/z3/blob/master/examples/python/parallel.py
24. Z3 C++ Context API — https://z3prover.github.io/api/html/classz3_1_1context.html
25. Z3 Spacer API — https://github.com/Z3Prover/z3/blob/master/src/api/z3_spacer.h
26. Microsoft Research, *SPACER and Z3* — https://www.microsoft.com/en-us/research/blog/spacer-and-z3-accessible-reliable-model-checking-as-theorem-proving/
27. *Evaluating Z3's Performance on Real Number Constraints*（TU Delft）— https://repository.tudelft.nl/file/File_dde6c24a73-4078-a900-255bb7f95ff9
28. Z3 issue #1141（push/pop 性能）— https://github.com/Z3Prover/z3/issues/1141
29. Z3 issue #7441（增量求解）— https://github.com/Z3Prover/z3/issues/7441
30. SMT-LIB logics — https://smt-lib.org/logics.shtml
31. *Undecidability and hardness in mixed-integer nonlinear programming* — https://www.lix.polytechnique.fr/~liberti/rairo18.pdf
32. *Survival Guide to Presburger Arithmetic* — https://www.cs.ox.ac.uk/people/christoph.haase/home/publication/haa-18/haa-18.pdf
33. Bagnara et al., *The Automatic Synthesis of Linear Ranking Functions* — https://www.cs.unipr.it/~bagnara/Papers/PDF/IC10.pdf
34. *Relational Constraint Solving in SMT* — http://theory.stanford.edu/~barrett/pubs/MRT+17.pdf
35. *Reducing OWL entailment to description logic satisfiability*（检索资料）— https://scispace.com/pdf/reducing-owl-entailment-to-description-logic-satisfiability-ncjiytfq1o.pdf
36. *A Petri Net Approach for the Modeling and Analysis of Video Games*（检索入口）— https://www.researchgate.net/publication/220800018_A_Petri_Net_Approach_for_the_Modeling_and_Analysis_of_Video_Games
37. *DEv-PROMELA*（形式化游戏模型检查）— https://journals.sagepub.com/doi/10.1177/0037549720946107
38. *Prototyping Games with BIPED* `[unverified]` — https://exa.ai/library/publication/8xxppn2qmkh
39. Lean / mathlib4 — https://github.com/leanprover-community/mathlib4
40. Z3 仓库 — https://github.com/Z3Prover/z3